



Due Date: 23:59 on Thursday, April 16th, 2026

Traffic Sign Detection using Hough Transform and RANSAC

In this assignment, you will build a traffic sign detection pipeline. First, you will extract edge points from images using the Canny edge detector. Then, you will detect circular traffic signs using both using the Hough Transform and RANSAC. An example of the pipeline is shown in Figure 1.

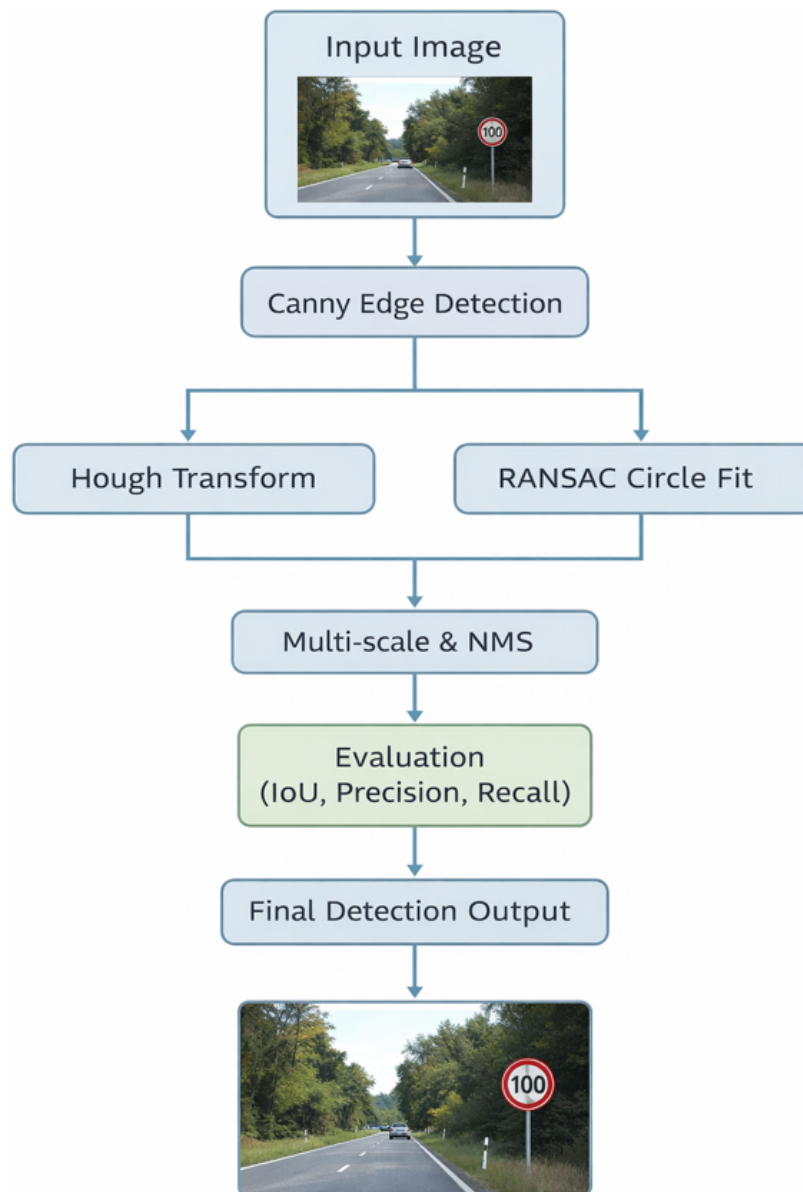


Figure 1: Overview of the traffic sign detection pipeline

Canny Edge Detector, Hough Transform, RANSAC

Canny Edge Detector: The Canny edge detector is a multi-stage algorithm that produces thin, well-localised edge maps. Its pipeline consists of: (1) Gaussian smoothing to suppress noise, (2) gradient magnitude and direction computation, (3) non-maximum suppression to thin edges to single-pixel width, and (4) hysteresis linking to retain only strong, connected edges. The two hysteresis thresholds (low and high) and the Gaussian blur standard deviation σ are the primary tunable parameters and significantly affect both edge map quality and detection performance.

Hough Transform: The Hough Transform is a voting-based method used to detect parameterized shapes in edge maps. Each edge point casts votes into an accumulator array; peaks in the accumulator correspond to shape instances.

RANSAC: Random Sample Consensus (RANSAC) is an iterative, robust model-fitting algorithm. It repeatedly (1) samples the minimum number of points needed to fit a model, (2) counts inliers and (3) keeps the model with the most inliers.

Both Hough Transform and RANSAC methods address the same problem—fitting shapes under noise and outliers—but they have different computational trade-offs that you will study empirically.

Dataset

You will use a subset of the German Traffic Sign Detection Benchmark (GTSDDB). The dataset consists of real-world road images containing traffic signs of different shapes such as circular, triangular, and rectangular signs.

Each image may contain one or more traffic signs. A simplified subset of the dataset is provided for educational purposes. The dataset can be accessed at https://drive.google.com/file/d/1JkmXQa26DGzU8ze055G4F7_r-ziZNj7I/view?usp=sharing.

The dataset is organised as follows:

- `Dataset_simplified/images` — input images (.ppm format)
- `Dataset_simplified/gt.txt` — ground-truth annotations. Each line follows the format: `image_name; x_min; y_min; x_max; y_max; class_id`
- `Dataset_simplified/README.txt` — description of the dataset



Figure 2: Sample images from the dataset

Implementation Details

1. All code must be written in Python 3. You may use `cv2.Canny` for edge detection. You may also use NumPy, SciPy, Matplotlib, and OpenCV for image I/O and visualisation. However, the Hough Transform and RANSAC must be implemented from scratch — calling `cv2.HoughCircles` or any library RANSAC function as a substitute for your own implementation is not permitted.
2. The entire implementation and analysis must be contained in a single Jupyter Notebook, which serves as both your code submission and your report.
3. **Canny edge detection[8pts]:** You may use `cv2.Canny` for edge detection. Your task is to understand its parameters and optimise them for the traffic sign detection problem. Conduct a parameter sensitivity study:

- Fix the high threshold and vary the low threshold; show and discuss how the edge map changes.
 - Fix both thresholds and vary the Gaussian blur kernel size applied before calling `cv2.Canny`; show the effect on edge density and continuity.
 - For each configuration, also report the detection F1-score so that the link between parameter choices and high-level performance is explicit.
 - State and justify your final chosen parameter values.
 - You are expected to understand what each Canny parameter controls conceptually. Your notebook should include corresponding discussions in a Markdown cell.
4. **Hough Transform [22pts]:** Implement your own Hough Transform to detect circular traffic signs. Your implementation must:
 - Build and fill a 3-D accumulator array over (x_c, y_c, r)
 - Support a configurable radius search range $[r_{\min}, r_{\max}]$.
 - Apply non-maximum suppression in the accumulator to suppress duplicate detections.
 - Return detected circles as $(x_c, y_c, r, \text{vote_count})$ tuples.
 5. **RANSAC [20pts]:** Implement RANSAC for circle fitting from edge points:
 - Minimum sample size is 3 points. Fit a circle analytically to each sample.
 - Count inliers: edge points within distance tolerance ϵ of the circle boundary.
 - The number of iterations N and the inlier tolerance ϵ are configurable parameters.
 6. **Quantative Comparison [20pts]:** Compare Hough Transform and RANSAC in terms of runtime and detection rate. You can present the metrics below in a summary comparison table:
 - IoU (Intersection over Union) between each detected circle and the nearest ground-truth annotation.
 - Precision, Recall, and F1-score at IoU threshold 0.5.
 - Precision–Recall curve obtained by sweeping the vote/inlier threshold
 - Runtime (seconds per image, averaged over the dataset).
 7. **Multiscale Detection and NMS [15pts]:** Traffic signs appear at varying distances and therefore at different scales. Therefore, you need to apply Multi-Scale Detection and Non-Maximum Suppression. Extend your pipeline as follows:
 - Build an image pyramid with at least 3 scales.
 - Run Canny edge detection and your shape detectors at each scale independently.
 - Map all detections back to the original image coordinate space.
 - Apply Non-Maximum Suppression (NMS) using IoU overlap to merge duplicate detections across scales. Report how NMS affects precision and recall.
 8. **Failure Analysis and Discussions [15pts]:** Visualize all your results by plotting detected shapes on original images. Visualize failure cases. Discuss thoroughly the pros and cons of the approaches.

The Notebook (Code + Report)

The notebook must be fully runnable from top to bottom with a clean kernel. Organise it with the following clearly labelled sections:

1. **Edge Detection.** Canny parameter optimisation study with at least 10 edge map visualisations. A Markdown explanation of the Canny algorithm stages and a justification of your final parameter choices, linked to F1-score of sign detection.
2. **Shape Detection.** Hough Circle Transform and RANSAC implementations, each accompanied by inline pseudo-code in a Markdown cell.
3. **Quantitative Evaluation.** Summary table and Precision–Recall curves comparing Hough and RANSAC. Discussion of which method is preferable and why.

4. **Multi-Scale Detection.** Image pyramid construction, per-scale detections, NMS, and a before/after precision–recall comparison.
5. **Failure Analysis and Discussions.** At least 5 failure cases. For each: show the image, identify the cause (occlusion, scale, lighting, etc.), and propose a possible fix.

NOTE: All figures must be generated inline inside the notebook. Do not link to external image files.

What to Hand In

Your submission format will be:

- b<studentNumber>.ipynb (your Jupyter Notebook, fully executed with all cell outputs visible)

Submission will be done via <https://submit.cs.hacettepe.edu.tr>.

Do not include the dataset folder in your submission. A separate report.pdf is not required—the notebook is your report.

Academic Integrity

All work on assignments must be done individually unless stated otherwise. You are encouraged to discuss the assignments given with your classmates, but these discussions should be conducted abstractly. That is, discussions related to a particular solution to a specific problem (either in actual code or in pseudocode) will not be tolerated. In short, turning in someone else’s work, in whole or in part, as your own work will be considered a violation of academic integrity. Please note that the former condition also holds for the material found on the web, as everything on the web has been written by someone else.

References

- [1] <https://sid.erda.dk/public/archives/ff17dc924eba88d5d01a807357d6614c/published-archive.html>